

CN3104 Cheatsheet AY25/26

Github:Shaunteojiwu
LinkedIn

NONLINEAR EQUATIONS

Goal: Solve $f(x) = 0$.

Graphical Method

-Plot $f(x)$ vs x ; roots where curve crosses x-axis.
-Simple but imprecise.

Incremental Search

-Using computer's brute force (relies on trying all possibilities)
1) Divide $[x_{\min}, x_{\max}]$ into n intervals.
2) Find sign changes: if $f(x_i)f(x_{i+1}) < 0$, a root lies inside.
-Use MATLAB function 'incsearch'
-Default number of intervals, ns=50

Disadvantages: -Misses roots if step too large; slow if step too small.

Bracketing Method:Bisection

1) Guess 2 initial starting point, where the root is within.
 $f(a)f(b) < 0$ (1 positive, 1 negative-can apply incremental search/graphical method for initial guess).
2) Iterate midpoint:

$$c = \frac{a+b}{2}.$$

3) If $f(a)f(c) < 0$ set $b = c$, else $a = c$. 4) Stop when stopping criterion met. (threshold error, similar to error in fixed-point iteration)
Guaranteed convergence, slow.

Fixed-Point Iteration

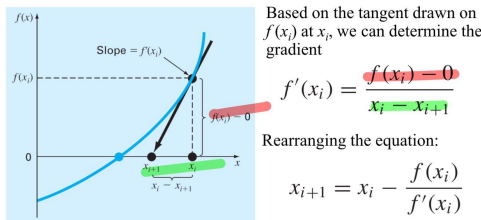
1) Rewrite $x = g(x)$.
2) Iterate $x_{i+1} = g(x_i)$.
3) Calculate error, Error:

$$e_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| 100\%.$$

-Stop when error within 0.01 -For bisection, error x_{i+1} is new root for the present iteration

Newton-Raphson

Open Method: Newton-Raphson



Newton-Raphson iterative equation

-Fast convergence.
-Requires derivative and good initial guess.
-Conditions for stop same as secant method.

Secant Method

No derivative needed, uses backward finite difference to estimate $f'(x)$:

$$x_{i+1} = x_i - f(x_i) \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}.$$

-Requires 2 starting value (do not need to bracket root)
-Conditions for stop: $x_i \approx \text{constant}$ or $f(x_i) \approx 0$

SYSTEMS OF LINEAR EQUATIONS

Solve $A\mathbf{x} = \mathbf{b}$.

Gaussian Elimination vs. LU Factorization

Gaussian Elimination (GE): Goal: convert $A \rightarrow U$ (upper triangular).
Uses row operations:

$$R_2 \leftarrow R_2 - \frac{a_{21}}{a_{11}} R_1.$$

Matrix A is modified during elimination. Does not store multipliers.

LU Factorization: Goal: write $A = LU$. U is obtained from elimination (same as GE). L stores the multipliers used in elimination. To avoid changing A , start L as the identity matrix and insert multipliers.

Identity Matrix Trick:

$$L = I \quad \Rightarrow \quad L(2,1) = m_{21} = \frac{a_{21}}{a_{11}}$$

Insert each multiplier into L while forming U .

Example:

$$A = \begin{bmatrix} 2 & 4 \\ 6 & 5 \end{bmatrix}, \quad m_{21} = \frac{6}{2} = 3$$

$$U = \begin{bmatrix} 2 & 4 \\ 0 & -7 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$$

$$A = LU$$

Forward elimination to make U upper triangular. Then back-substitute.

Pivot equation: eliminate variables below pivot using

$$\text{factor} = \frac{a_{ik}}{a_{kk}}.$$

Ill-Conditioning

Small changes in data cause large changes in solution.

LINEAR REGRESSION

Given data (x_i, y_i) .

Simple Linear Regression

Goal: Fit a straight line that best predicts y from x . A model is linear if it is linear in the coefficients (parameters). Model:

$$y = a_0 + a_1 x + e.$$

Normal equations:

$$a_0 n + a_1 \sum x_i = \sum y_i$$

$$a_0 \sum x_i + a_1 \sum x_i^2 = \sum x_i y_i.$$

General Least Squares

Matrix form:

$$\hat{y} = Xb$$

$$\varepsilon = y - Xb$$

Minimize $\Phi = \varepsilon^T \varepsilon$.

Solution:

$$b = (X^T X)^{-1} X^T y.$$

NUMERICAL INTEGRATION

Goal:

$$I = \int_a^b f(x) dx.$$

Trapezoidal Rule

Single interval:

$$I \approx \frac{b-a}{2} [f(a) + f(b)].$$

Composite:

$$I = \frac{h}{2} [f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)].$$

Error:

$$E_a = -\frac{(b-a)^3}{12n^2} f''(\xi).$$

Simpson's 1/3 Rule

Requires even number of intervals:

$$I = (b-a) \frac{f(x_0) + 4f(x_1) + f(x_2)}{6}$$

Composite:

$$I = (b-a) \frac{f(x_0) + 4 \sum_{i \text{ odd}}^{n-1} f(x_i) + 2 \sum_{j=2}^{n-2} f(x_j) + f(x_n)}{3n}$$

Simpson's 3/8 Rule

$$I = (b-a) \frac{f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)}{8}$$

NUMERICAL DIFFERENTIATION

Using finite differences.

Forward Difference

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h}.$$

Error $O(h)$.

Backward Difference

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{h}.$$

Error $O(h)$.

Central Difference

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}.$$

Error $O(h^2)$.

ODEs

Solve:

$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$

Euler Method

Use finite difference approximation

$\frac{dv}{dt} \approx \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$

Rearranging the expression

$v_{i+1} = v_i + \frac{dvi}{dt} \Delta t$

General form:

$y_{i+1} = y_i + f(t_i, y_i) \, h$

Heun’s Method (Improved Euler)

$y_1 = y(t_0) + \frac{h}{2} \Big(f(t_0, y_0) + f\big(t_1, \, y_0 + hf(t_0, y_0)\big) \Big)$

Predictor:

$k_1 = f(t_i, y_i)$

Corrector:

$k_2 = f(t_i + h, y_i + hk_1)$

Update:

$y_{i+1} = y_i + \frac{h}{2} (k_1 + k_2).$

Midpoint Method

$y_{i+1} = y_i + f\Big(t_{i+\frac{1}{2}}, \, y_{i+\frac{1}{2}}\Big) \, h$

$k_1 = f(t_i, y_i)$

$k_m = f(t_i + \frac{h}{2}, y_i + \frac{h}{2} k_1)$

$y_{i+1} = y_i + hk_m.$

Runge-Kutta 4 (RK4)

$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)h$

$k_1 = f(t_i, y_i)$

$k_2 = f\Big(t_i + \frac{h}{2}, \, y_i + \frac{h}{2} k_1\Big)$

$k_3 = f\Big(t_i + \frac{h}{2}, \, y_i + \frac{h}{2} k_2\Big)$

$k_4 = f(t_i + h, y_i + hk_3)$

Adaptive RK (ode45)

MATLAB uses RK4(5) pair. Small step when function changes quickly; large step otherwise.